

1

1.1 Sizeof und Arrays

a)

Das Array wird mit vom Typ `int` Initialisiert. Eine Variable vom Typ `int` benötigt 4 Byte Speicherplatz. Somit braucht das Array insgesamt $4 \text{ Byte} \cdot 8 = 32 \text{ Byte}$ Speicherplatz. 0.5/0.5

b)

`sizeof(a)` gibt die Gesamtgröße des Arrays in Byte zurück. Siehe a)
=> es ergibt sich eine Gesamtgröße von 32 Byte.

Um die Anzahl der Elemente im Array zu berechnen kann man die Gesamtgröße des Arrays durch die Größe eines einzelnen Elements teilen: `sizeof(a) / sizeof(a[0])`. Dies ergibt $32 \text{ Byte} / 4 \text{ Byte} = 8$ Elemente.

```
int a [8];
int i;

for (i = 0; i < sizeof(a) / sizeof(a[0]); i++) {
    a[i] = 1;
}
```

1/1

c)

Der das Ergebnis wird 8 sein, da die Operation die Speichergröße des Datentyps des Pointers zurück gibt. 0.5/0.5

d)

Ein möglichkeit wäre die Übergabe der Arraygröße als zusätzlichen Parameter an die Funktion, da in C "Array Decay" ein

Array in einen Pointer umwandelt, wenn es an eine Funktion übergeben wird.

1/1

richtig, aber wie würde die konkrete Berechnung aussehen

1.2

a)

Zeit (ms)	Ereignis	Laufender Prozess	A	B	C	A Laufzeit	B Laufzeit	C Laufzeit	Ready-Queue	HDD-Queue (35 ms)	USB-Queue (45 ms)
Start	-	-	Rechenbereit	Rechenbereit	Rechenbereit	0	0	0	A, B, C	-	-
0	Scheduler wählt A	A	Rechnend	Rechenbereit	Rechenbereit	5			B, C	-	-
5	-	A	Rechnend	Rechenbereit	Rechenbereit	10			B, C, A	-	-
10	Scheduler wählt B	B	Rechenbereit	Rechnend	Rechenbereit		5		C, A	-	-
15	B blockiert, C wird gewählt	C	Rechenbereit	Blockiert	Rechnend			5	A	-	B
20	-	C	Rechenbereit	Blockiert	Rechnend			10	C	-	B
25	Scheduler wählt A	A	Rechnend	Blockiert	Rechenbereit	15			C	-	B
30	A blockiert, C wird gewählt	C	Blockiert	Blockiert	Rechnend			15	-	A	B
35	-	C	Blockiert	Blockiert	Rechnend			20	C	A	B
40	Scheduler wählt C	C	Blockiert	Blockiert	Rechnend			25	-	A	B
45	-	C	Blockiert	Blockiert	Rechnend			30	C	A	B
50	Scheduler wählt C	C	Blockiert	Blockiert	Rechnend			35	-	A	B
55	-	C	Blockiert	Blockiert	Rechnend			40	B, C	A	B
60	Scheduler wählt B	B	Blockiert	Rechnend	Rechenbereit		10		C	A	-
65	-	B	Rechenbereit	Rechnend	Rechenbereit		15		C, A	-	-
70	Scheduler wählt C	C	Rechenbereit	Rechenbereit	Rechnend			45	A, B	-	-
75	-	C	Rechenbereit	Rechenbereit	Rechnend			50	A, B	-	-
80	Scheduler wählt A	A	Rechnend	Rechenbereit	Done	20			B	-	-
85	A blockiert, B wird gewählt	B	Blockiert	Rechnend	Done		20		-	-	A
90	-	B	Blockiert	Rechnend	Done		25		B	-	A
95	Scheduler wählt C	B	Blockiert	Rechnend	Done		30		-	-	A
100	-	B	Blockiert	Rechnend	Done		35		-	-	A
105	Scheduler wählt B	B	Blockiert	Rechnend	Done		40		-	-	A
110	-	B	Blockiert	Rechnend	Done		45		-	-	A
115	Scheduler wählt B	B	Blockiert	Rechnend	Done		50		-	-	A
120	B ist done, Scheduler wartet	-	Blockiert	Done	Done				-	-	A
125	-	-	Blockiert	Done	Done				A	-	A
130	Scheduler wählt A	A	Rechnend	Done	Done	25			A	-	-
135	-	A	Rechnend	Done	Done	30			-	-	-
140	Scheduler wählt A	A	Rechnend	Done	Done	35			A	-	-
145	-	A	Rechnend	Done	Done	40			-	-	-
150	Scheduler wählt A	A	Rechnend	Done	Done	45			A	-	-
155	-	A	Done	Done	Done	50			-	-	-

3/3

1.3

a)

Header	Meaning	Erklärung
USER	User	Der Benutzername des Eigentümers des Prozesses
PID	Process ID	Die eindeutige Identifikationsnummer des Prozesses
PPID	Parent Process ID	Die Prozess-ID des übergeordneten Prozesses
%CPU	CPU usage	Der prozentuale Anteil der CPU-Zeit, den der Prozess seit seinem Start verwendet hat

Header	Meaning	Erklärung
%MEM	Memory usage	Der prozentuale Anteil des gesamten physischen Arbeitsspeichers, den der Prozess nutzt
TT	Terminal	Das Terminal, mit dem der Prozess verknüpft ist
STAT	Process state	Der aktuelle Zustand des Prozesses
STARTED	Start time/date	Wann der Prozess gestartet wurde
TIME	CPU time	Die gesamte CPU-Zeit, die der Prozess bisher verwendet hat
CMD	Command	Der Befehl (und seine Argumente), mit dem der Prozess gestartet wurde

1/1

b)

Der Prozess 3787 hat den Statuscode S+, welcher sich wie folgt aufschlüsseln lässt:

S: interruptible sleep (waiting for an event to complete)

+: is in the foreground process group

- Der Prozess schläft und wartet auf Ereignisse (evt. Benutzereingabe)
- Er ist interaktiv und gehört zur ordergrundgruppe des Terminals

1/1

c)

```
-- 0956  root      login -- mschroetter
\-- 1009  mschroe  -zsh
  \-- 1929  mschroe  tmux new-session
    \-- 3828  mschroe  -zsh
      \--- 3967  mschroe  ps -eo user,pid,ppid
```

1/1

Aufgabe 1.4: 2/3

print_file.c

```
#include <stdlib.h>
#include <fcntl.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>

#define SIZE 512

int main() {

    int fd = open("./test.txt", O_RDWR);
    if (fd == -1){
        perror("could not open file");
        return EXIT_FAILURE;
    }

    char * buffer = (char *) calloc(SIZE, sizeof(char));

    if (!buffer) {
        perror("could not allocate memory");
        close(fd);
        return EXIT_FAILURE; ✓
    }

    ssize_t r = read(fd, buffer, SIZE-1);

    if (r == -1) {
        perror("could not read file");
        free(buffer);
        close(fd);
        return EXIT_FAILURE;
    }

    printf("%.*s", SIZE-1, buffer);
    free(buffer);
    close(fd);
    return EXIT_SUCCESS;
}
```

workdir.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

char *mywdfunc(unsigned int bufsize) {
    if (bufsize < 100) {
        return NULL;
    }

    char *buf = (char *)calloc(bufsize, sizeof(char));
    if (buf) {
        char *cwd = getcwd(buf, bufsize-1);
        if (!cwd) {
            free(buf);
            return NULL;
        }
    }

    return buf;
}

int main() {
    char *ret = mywdfunc(512);

    printf("%s", ret);
    free(ret);
    return EXIT_SUCCESS;
}
```

Rückgabewert von mywdfunc() ebenfalls auf NULL prüfen, um Fehlerfall zu erkennen. -1

Aufgabe 1.5: 6/7

dl_proc_list.h 1/1

```
typedef struct dl_proc_node {  
    int pid;  
    int ppid;  
    struct dl_proc_node* prev; ✓  
    struct dl_proc_node* next;  
} dl_proc_node;
```

```
typedef struct dl_proc_list {  
    dl_proc_node* head; ✓  
} dl_proc_list;
```

```
dl_proc_list* dl_proc_list_create(void);
```

```
int dl_proc_list_insert(dl_proc_list *list, int pid);
```

```
int dl_proc_list_get(dl_proc_list *list, int position, int* ✓  
pid, int* ppid);
```

```
int dl_proc_list_remove(dl_proc_list *list, int pid);
```

```
void dl_proc_list_free(dl_proc_list *list);
```

Makefile vorhanden. 0,5/0,5

dl_proc_list.c 3/3,5

```
#include "dl_proc_list.h"
#include <stdlib.h>

// Creates a new process list with the initial process (PID = 1, PPID = 0)
dl_proc_list* dl_proc_list_create() {
    // Create list
    dl_proc_list *list;
    list = (dl_proc_list *) malloc(sizeof(dl_proc_list));

    if (!list) {
        return NULL; ✓
    }

    // Create first process node
    dl_proc_node *first_node;
    first_node = (dl_proc_node *)
malloc(sizeof(dl_proc_node));

    if (!first_node) {
        free(list); ✓
        return NULL;
    }

    // Initialize first process node (PID = 1, PPID = 0)
    first_node->pid = 1;
    first_node->ppid = 0; ✓

    first_node->prev = NULL;
    first_node->next = NULL; ✓

    // Set head of list to first node
    list->head = first_node; ✓

    return list;
}

// Inserts a new process with given PID into process list
// Returns 1 on success, -1 on memory allocation failure,
// -2 on invalid args, -3 if PID already exists
int dl_proc_list_insert(dl_proc_list *list, int pid){
    // Invalid list
    if (!list) {
        return -2; ✓
    }

    // Invalid PID
    if (pid <= 1) {
        return -2; ✓
    }
}
```

```

    }

    // Find position to insert new process node
    dl_proc_node *pre_new = list->head;

    while (pre_new && pre_new->next && pre_new->next->pid <
pid) {
        pre_new = pre_new->next;
    }

    // PID already exists
    if ((pre_new && pre_new->pid == pid) ||
        (pre_new && pre_new->next && pre_new->next->pid ==
pid)) {
        return -3; ✓
    }

    // pre_new is either the end of the list or the last node
    with PID < new PID

    // Invalid position
    if (!pre_new) {
        return -3;
    }

    // Create new process node
    dl_proc_node *new_node = (dl_proc_node *)
malloc(sizeof(dl_proc_node));
    if (!new_node) {
        return -1; ✓
    }

    new_node->pid = pid; ✓

    if (pre_new->next) {
        new_node->ppid = pre_new->next->pid; // smaller
        inserted before larger
    } else {
        new_node->ppid = pre_new->pid; // larger inserted
        after smaller
    }

    new_node->prev = pre_new;
    new_node->next = pre_new->next; ✓

    if (pre_new->next) {
        pre_new->next->prev = new_node;
    }
    pre_new->next = new_node; ✓

```

Es können weitere Elemente in der Liste existieren, deren PID größer ist als von pre_new->next.

Liefert nicht notwendigerweise die bisher höchste Prozess-ID. -0.5

```

    return 0;
}

int dl_proc_list_get(dl_proc_list *list, int position, int*
pid, int* ppid){
    // Invalid list
    if (!list) {
        return -1; ✓
    }

    dl_proc_node *current = list->head;
    int index = 0;

    while (current && index < position) {
        current = current->next;
        index++; ✓
    }

    // Position out of bounds
    if (!current) {
        return -2; ✓
    }

    *pid = current->pid;
    *ppid = current->ppid; ✓

    return 0;
}

```

```

int dl_proc_list_remove(dl_proc_list *list, int pid){
    // Invalid list
    if (!list) {
        return -2; ✓
    }

    // Invalid PID, cannot remove PID 1
    if (pid <= 1) {
        return -2; ✓
    }

    dl_proc_node *current = list->head;

    while (current && current->pid != pid) {
        current = current->next; ✓
    }

    // PID not found
    if (!current) {
        return -3; ✓
    }
}

```

```

}

int parent_pid = current->ppid;

// Remove the node from the list
if (current->prev) {
    current->prev->next = current->next;
} else {
    // Removing head node ✓
    list->head = current->next;
}

if (current->next) {
    current->next->prev = current->prev; ✓
}

free(current); ✓
dl_proc_node *node = list->head;
while (node) {
    if (node->ppid == pid) {
        node->ppid = parent_pid; ✓
    }
    node = node->next;
}

return 0;
}

void dl_proc_list_free(dl_proc_list *list){
    if (!list) {
        return; ✓
    }

    dl_proc_node *current = list->head;
    dl_proc_node *next_node;

    while (current) {
        next_node = current->next;
        free(current); ✓
        current = next_node;
    }

    free(list); ✓
}

```

proctable.c 1,5/2

```
#include <stdio.h>
#include <stdlib.h>
#include "dl_proc_list.h"

int main(int argc, char *argv[]) {
    dl_proc_list *plist = dl_proc_list_create();
    if (!plist) {
        fprintf(stderr, "Fehler: Konnte Prozesstabelle nicht
erstellen.\n");
        return EXIT_FAILURE; ✓
    }

    for (int i = 1; i < argc; i++) {
        char *endptr;
        long val = strtol(argv[i], &endptr, 10);

        if (*endptr != '\0') {
            fprintf(stderr, "Warnung: Ungueltiges Argument
\"%s\" ignoriert.\n", argv[i]); ✓
            continue;
        }

        if (val == 0) {
            fprintf(stderr, "Warnung: PID = 0 wird ignoriert.
\n");
            continue; ✓
        }

        if (val > 0) {
            if (dl_proc_list_insert(plist, (int)val) < 0) {
                fprintf(stderr, "Fehler beim Einfuegen: PID
%d existiert bereits.\n", val); ✓
            }
        } else {
            int pid_to_remove = (int)(-val);
            if (pid_to_remove == 1) {
                fprintf(stderr, "Fehler: Prozess mit ID = 1
kann nicht geloescht werden.\n"); ✓
            } else if (dl_proc_list_remove(plist,
pid_to_remove) < 0) {
                fprintf(stderr, "Fehler beim Loeschen: Prozess
mit ID = %d existiert nicht.\n", pid_to_remove); ✓
            }
        }
    }

    int pos = 0, pid, ppid;
    while (dl_proc_list_get(plist, pos, &pid, &ppid) == 0) {
        printf("PID = %d, Parent PID = %d\n", pid, ppid); ✓
        pos++;
    }
}
```

strtol() liefert einen Wert vom Typ long zurück. Vor dem Casting nach int darauf achten, dass der Wert nicht unter INT_MIN bzw. über INT_MAX liegt. -0,5

```
}  
    dl_proc_list_free(plist);  
    return EXIT_SUCCESS; ✓  
}
```